

COMPSCI 589 Final Project - Spring 2026

Due **May 8**, 11:55 pm Eastern Time

Introduction

- Each group member was responsible for evaluating the performance of at least one unique algorithm implemented during the semester on each dataset
- The algorithms tested in this project were chosen from the pool of: k -NN, Decision Trees, multinomial Naive Bayes, Random Forests, and Neural Networks
- All work was done in the shared github repository https://github.com/19851xw/589_final_project

-
- **Katherine** trained the neural networks algorithm on all four datasets. She tested three hyperparameters: the architecture, the λ (regularization) parameter, and the α (learning rate) parameter.
 - **Tiffany** implemented KNN and Random Forest. She evaluated the hyperparameters k for kNN and n_{tree} for random forest.
 - **Xing-Wei** implemented Decision Trees. She evaluated the hyperparameters max depth and minimum split size.

Experiments and Analyses

1 The Hand-Written Digits Recognition Dataset

1.1 Neural Networks Algorithm

1.2 KNN

k	Accuracy	F1 Score
1	0.988571	0.989391
3	0.986857	0.987597
5	0.986286	0.987128
7	0.986286	0.987206
9	0.982857	0.984203
11	0.980571	0.982039
13	0.980000	0.980917
15	0.980571	0.982108
17	0.977143	0.978787
19	0.976000	0.977379
21	0.974857	0.976219
23	0.972571	0.974215
25	0.969714	0.971757
27	0.966286	0.968297
29	0.968571	0.970363
31	0.967429	0.968898
33	0.961714	0.963689
35	0.964571	0.966932
37	0.963429	0.965532
39	0.963429	0.965328
41	0.960571	0.962805
43	0.961143	0.962758
45	0.959429	0.961605
47	0.956571	0.959014
49	0.957714	0.960383
51	0.955429	0.958529



Figure 1: Handwritten Digits Accuracy Distribution

As k increases, the general trend for the performance is that it decreases. This is expected, as too large k values can lead to too generalized performance. Although, the overall performance is still generally high with the lowest accuracy and F1 score being around 0.96. The best configuration for this model based on this graph is at $k = 1$.

1.3 Random Forest

ntree	1	5	10	20	30	40	50
Accuracy	0.968223	0.969386	0.993427	1.0	0.996774	0.997059	1.0
Precision	0.954737	1.000000	1.000000	1.0	1.000000	1.000000	1.0
Recall	0.980000	0.934332	0.985000	1.0	0.993333	0.994118	1.0
F1 Score	0.962738	0.965294	0.992204	1.0	0.996552	0.996970	1.0

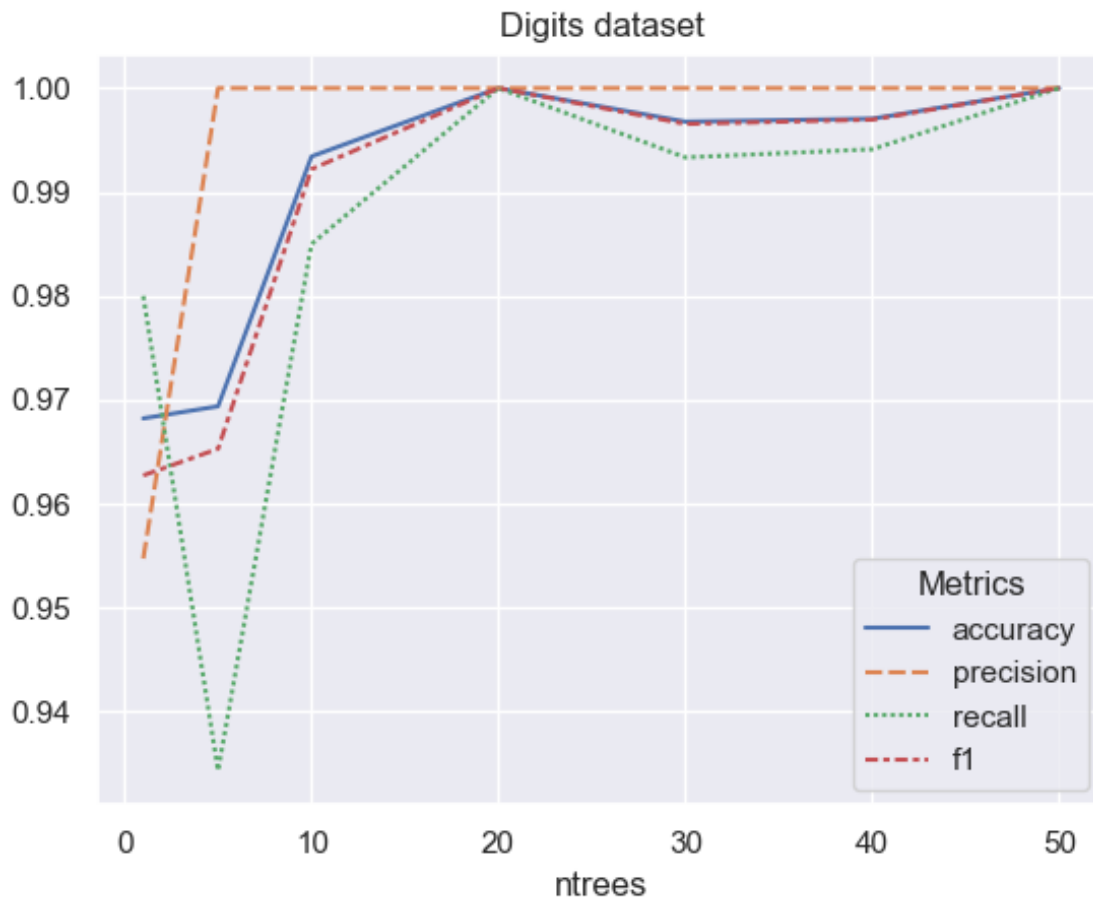


Figure 2: Digits Accuracy and F1 Score Distribution

Technically the highest accuracy and f1 occurs at $\text{ntree}=20$ and $\text{ntree}=50$ because the accuracy and f1 are both 1 with those values, but it is likely that the model is overfit. Because of this, it may be better to use a value of $\text{ntree}=5$ even though the performance is technically a little lower at an accuracy of 0.969386 and an f1 score of 0.965294.

2 The Oxford Parkinson's Disease Detection Dataset

2.1 Decision Trees Algorithm

We chose to use decision trees for this dataset because decision trees are best at finding precise boundaries when there are only numerical features. This is the case for the Parkinson's dataset, which has 22 numerical features and no categorical features. We use the hyperparameters of max depth and minimum split size to control the complexity of the decision tree. Below we tested 7 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 195 rows and 22 features.

Testing Depths: [2, 3, 5, 7, 9, 11, 13]

Testing Splits: [9, 19, 29, 39]

Depth	Split	Test Acc	Test F1
2	9	0.8503	0.8065
2	19	0.8117	0.7631
2	29	0.8454	0.7922
2	39	0.8625	0.7836
3	9	0.8681	0.8278
3	19	0.8312	0.7890
3	29	0.8309	0.7553
3	39	0.8620	0.7975
5	9	0.8575	0.8157
5	19	0.8556	0.8171
5	29	0.8362	0.7784
5	39	0.8611	0.7906
7	9	0.8456	0.8050
7	19	0.8211	0.7456
7	29	0.8312	0.7815
7	39	0.8617	0.7912
9	9	0.8323	0.7762
9	19	0.8515	0.8086
9	29	0.8534	0.8038
9	39	0.8512	0.7767
11	9	0.8562	0.7975
11	19	0.8220	0.7627
11	29	0.8273	0.7595
11	39	0.8606	0.7840
13	9	0.8628	0.8177
13	19	0.8361	0.7720
13	29	0.8348	0.7692
13	39	0.8495	0.7635

=====

BEST CONFIGURATION:

Max Depth: 3

Min Split: 9

F1 Score: 0.8278

=====

Final Results:

Training Accuracy: 0.9345

Training F1 Score: 0.9165

Testing Accuracy: 0.8508

Testing F1 Score: 0.8101

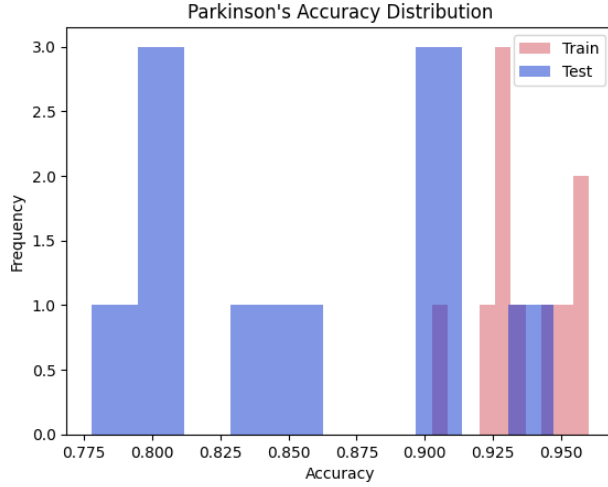


Figure 3: Parkinson's Accuracy Distribution

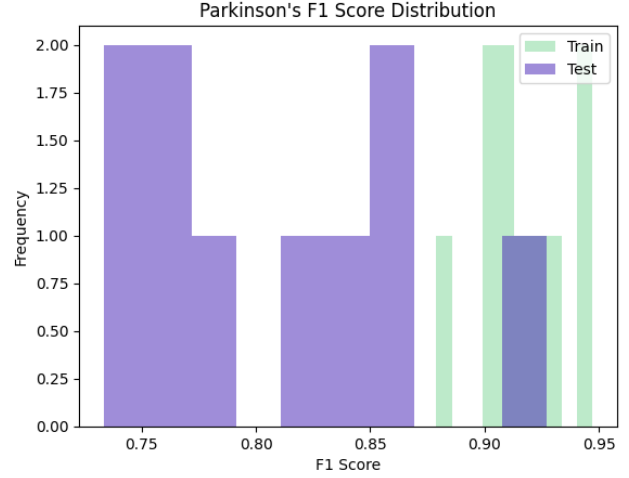


Figure 4: Parkinson's F1 Score Distribution

There is high variance on the testing scores, which may be due to the small size of the dataset making the model very sensitive to specific training data in each fold. The training scores are very concentrated around 0.93 which suggests high stability and low bias. There is a clear difference between the average training and testing scores, which suggest the model may be overfitting. Since the model is sensitive to specific training data, the model is less robust and may not generalize well to unseen data. Decision trees are well suited for numerical datasets, but when the sample size is small, the splits can become too specific to a training set which leads to the variance seen in testing.

2.2 KNN

k	Accuracy	F1 Score
1	0.966667	0.978033
3	0.944444	0.963738
5	0.944444	0.964514
7	0.944444	0.965475
9	0.927778	0.954590
11	0.900000	0.937926
13	0.872222	0.922714
15	0.861111	0.915862
17	0.850000	0.910674
19	0.838889	0.903946
21	0.811111	0.890880
23	0.827778	0.899946
25	0.850000	0.912620
27	0.838889	0.906579
29	0.838889	0.906613
31	0.844444	0.909247
33	0.866667	0.922270
35	0.872222	0.924905
37	0.872222	0.924716
39	0.850000	0.912842
41	0.855556	0.915269
43	0.833333	0.903978
45	0.844444	0.909831
47	0.822222	0.897769
49	0.827778	0.900780
51	0.816667	0.895134



Figure 5: Handwritten Digits Accuracy Distribution

2.3 Random Forest

ntree	1	5	10	20	30	40	50
Accuracy	0.805556	0.794444	0.816667	0.833333	0.850000	0.838889	0.850000
Precision	0.836293	0.843709	0.853481	0.853754	0.861730	0.854793	0.872728
Recall	0.935714	0.907143	0.928571	0.950000	0.964286	0.957143	0.950000
F1 Score	0.881610	0.872319	0.887543	0.898543	0.909364	0.902490	0.908122

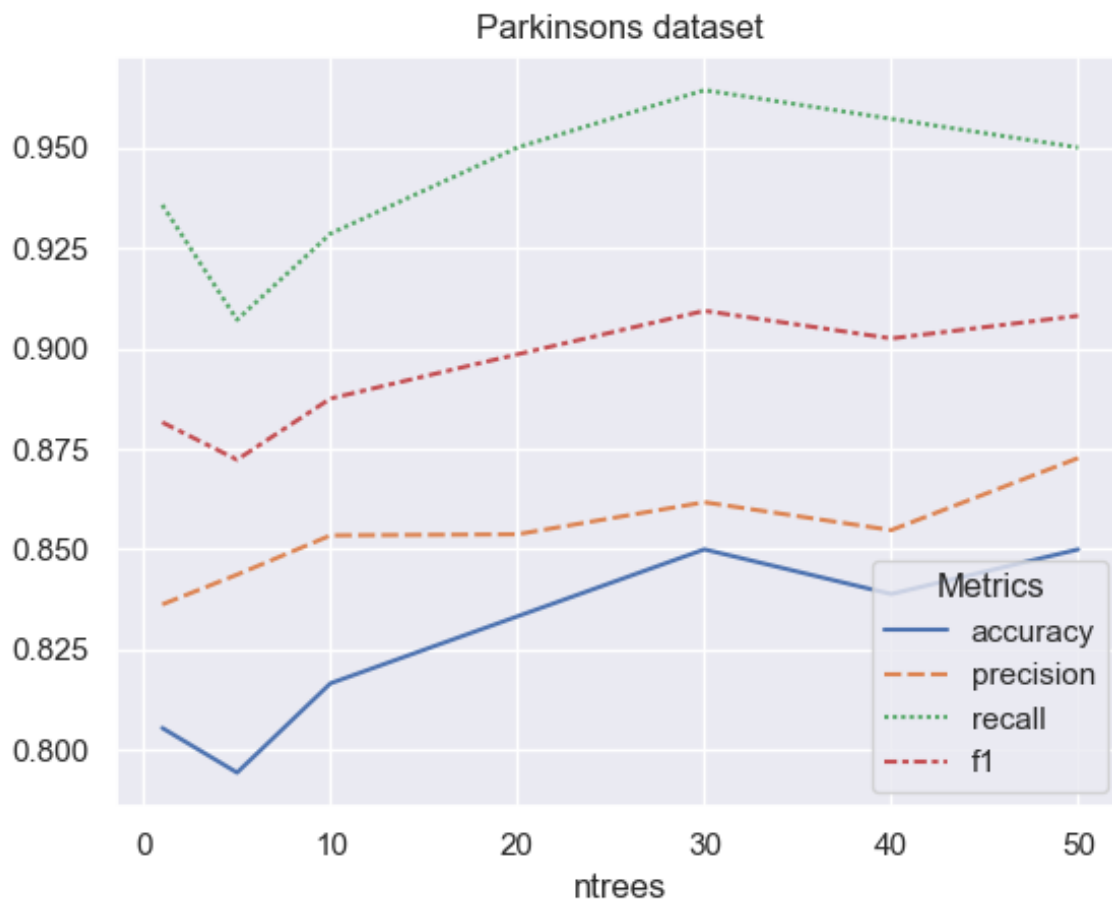


Figure 6: Parkinsons Accuracy and F1 Score Distribution

3 The Rice Grain Dataset

3.1 Neural Networks Algorithm

TODO

3.2 KNN

k	Accuracy	F1 Score
1	0.886614	0.867819
3	0.909974	0.894640
5	0.918898	0.904873
7	0.923885	0.910538
9	0.922047	0.908377
11	0.922835	0.909521
13	0.924409	0.911221
15	0.925984	0.913105
17	0.925984	0.912772
19	0.927034	0.914136
21	0.926509	0.913226
23	0.927034	0.914092
25	0.928084	0.915473
27	0.925984	0.912969
29	0.925459	0.912357
31	0.928346	0.915703
33	0.926509	0.913735
35	0.925984	0.912977
37	0.926509	0.913497
39	0.927559	0.915076
41	0.927034	0.914137
43	0.927559	0.914960
45	0.924934	0.911821
47	0.927559	0.914945
49	0.927559	0.914773
51	0.928084	0.915623

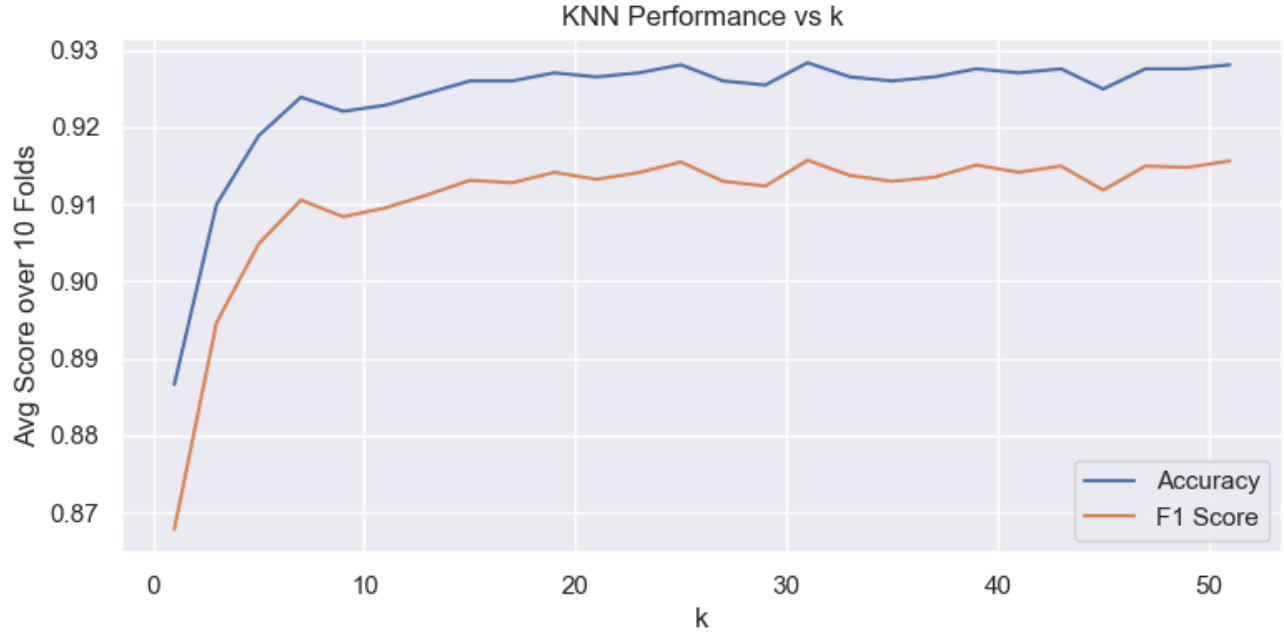


Figure 7: Rice Accuracy and F1 Score Distribution

3.3 Decision Trees Algorithm

We chose to test decision trees on the rice dataset because it has a moderate number of features (7) and a relatively large number of samples (3810), which allows us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below, where we tested 6 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 3810 rows and 7 features.

Testing Depths: [2, 3, 5, 7, 9, 11]

Testing Splits: [190, 381, 571, 762]

Depth	Split	Test Acc	Test F1
2	190	0.9215	0.9192
2	381	0.9228	0.9205
2	571	0.9226	0.9202
2	762	0.9239	0.9215
3	190	0.9234	0.9210
3	381	0.9213	0.9189
3	571	0.9202	0.9179
3	762	0.9239	0.9215
5	190	0.9213	0.9189
5	381	0.9234	0.9210
5	571	0.9226	0.9202
5	762	0.9239	0.9215
7	190	0.9223	0.9199
7	381	0.9226	0.9203
7	571	0.9215	0.9192
7	762	0.9239	0.9215
9	190	0.9234	0.9210
9	381	0.9220	0.9197

9	571	0.9239	0.9215
9	762	0.9239	0.9215
11	190	0.9223	0.9199
11	381	0.9220	0.9197
11	571	0.9218	0.9194
11	762	0.9239	0.9215

=====

BEST CONFIGURATION:

Max Depth: 11

Min Split: 762

F1 Score: 0.9215

=====

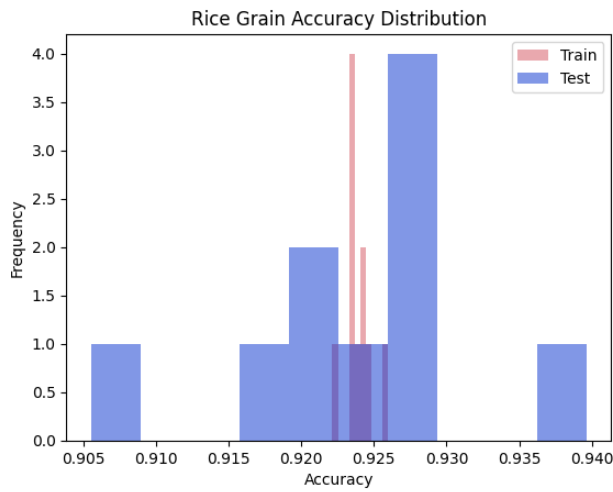


Figure 8: Rice Accuracy Distribution

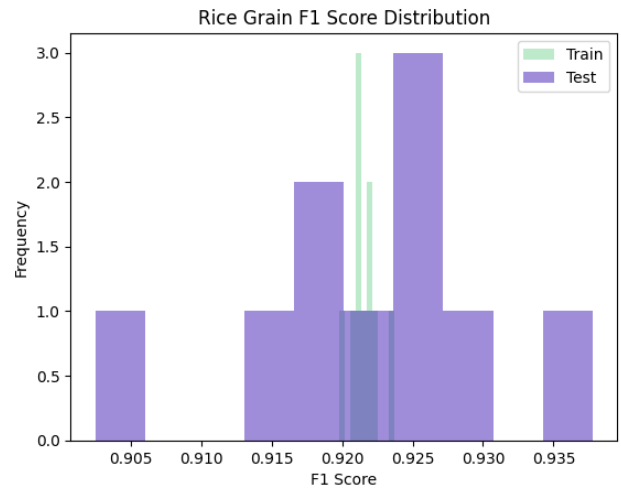


Figure 9: Rice F1 Score Distribution

Final Results:

Training Accuracy: 0.9239

Training F1 Score: 0.9216

Testing Accuracy: 0.9239

Testing F1 Score: 0.9215

For both models, the training scores are highly concentrated around 0.92, while the testing scores are more spread out, with a mean around 0.923 for accuracy and 0.922 for F1 score. The training performance suggests that the models have high stability and low bias, however, the testing performance indicates that there may be some sensitivity to the specific training data used in each fold, leading to a poor local optimum. Since the mean training scores are very close to the mean testing scores, it suggests that the models are robust and are not overfitting. The accuracy and F1 scores are also very close to each other which suggests that the models are performing well across all labels in the dataset and that the labels in the dataset are balanced. Decision trees can easily split numerical features to find precise boundaries, so they are well-suited for the rice dataset, which consists of numerical attributes.

4 The Credit Approval Dataset

4.1 Neural Networks Algorithm

TODO

4.2 KNN

k	Accuracy	F1 Score
1	0.806250	0.780158
3	0.859375	0.845093
5	0.864062	0.853219
7	0.873437	0.864596
9	0.862500	0.854968
11	0.865625	0.858536
13	0.856250	0.848834
15	0.860938	0.854310
17	0.862500	0.858192
19	0.864062	0.860247
21	0.865625	0.860856
23	0.857812	0.853120
25	0.864062	0.858874
27	0.867188	0.863106
29	0.857812	0.854402
31	0.860938	0.856263
33	0.868750	0.865578
35	0.857812	0.852417
37	0.862500	0.857252
39	0.864062	0.856573
41	0.860938	0.853287
43	0.856250	0.848844
45	0.862500	0.855567
47	0.860938	0.854175
49	0.864062	0.856989
51	0.856250	0.849728



Figure 10: Credit Approval Accuracy and F1 Score Distribution

4.3 Decision Trees Algorithm

We chose to test decision trees on the credit approval dataset because it has a moderate number of features (15) and a relatively small number of samples (653), which allows us to test two different hyperparameters; tree depths and minimum split sizes. This is shown in the tuning results below where we tested 6 different tree depths and 4 different minimum split sizes.

Tuning for dataset with 653 rows and 15 features.

Testing Depths: [2, 4, 6, 8, 10, 12]

Testing Splits: [32, 65, 97, 130]

Depth	Split	Test Acc	Test F1

2	32	0.8622	0.8621
2	65	0.8639	0.8636
2	97	0.8622	0.8622
2	130	0.8622	0.8620
4	32	0.8378	0.8358
4	65	0.8482	0.8473
4	97	0.8588	0.8580
4	130	0.8577	0.8566
6	32	0.8318	0.8293
6	65	0.8302	0.8288
6	97	0.8501	0.8491
6	130	0.8610	0.8606
8	32	0.8378	0.8356
8	65	0.8576	0.8568
8	97	0.8496	0.8489
8	130	0.8620	0.8612
10	32	0.8377	0.8356
10	65	0.8409	0.8397

10	97	0.8437	0.8423
10	130	0.8504	0.8495
12	32	0.8375	0.8350
12	65	0.8451	0.8439
12	97	0.8668	0.8654
12	130	0.8635	0.8626

```

=====
BEST CONFIGURATION:
Max Depth: 12
Min Split: 97
F1 Score: 0.8654
=====

```

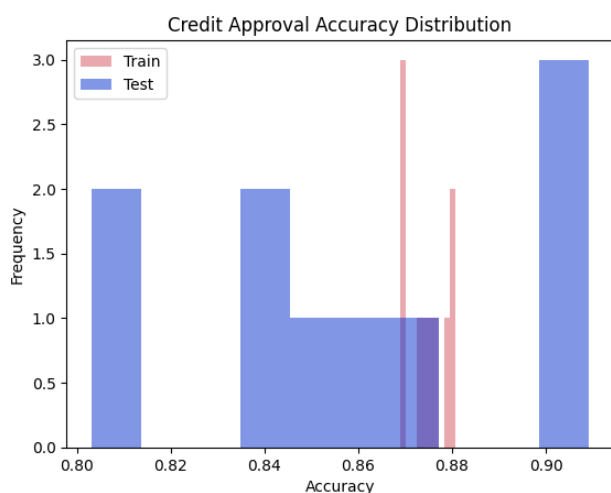


Figure 11: Credit Approval Accuracy Distribution



Figure 12: Credit Approval F1 Score Distribution

```

Final Results:
Training Accuracy: 0.8746
Training F1 Score: 0.8741
Testing Accuracy: 0.8606
Testing F1 Score: 0.8601

```

The test scores are widely spread which suggests the model is sensitive to the specific training data used in each fold, leading to a poor local optimum. Especially in smaller datasets, the presence/absence of a few instances in a training fold can have a significant impact on the tree structure and thus the performance. The training scores are more concentrated around 0.87, while some test folds outperform the training performance. This suggests that the model is stable during training, but is not consistent on generalizing depending on the test fold. The accuracy and F1 scores are also very close to each other which suggests that the model is performing well across all labels in the dataset and that the labels in the dataset are balanced. Since the credit approval dataset has some categorical features, they need to be factorized. If there are a lot of unique values in a categorical feature, it can lead to a large number of splits in the decision tree, which may be why there is more spread in the test scores for this dataset.

	Handwritten Digits		Parkinson's		Rice Grains		Credit Approval	
	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score	Accuracy	F1-Score
Neural Networks								
K-Nearest Neighbors	0.97109	0.97281	0.866239	0.92118	0.9237	0.91048	0.86009	0.85289
Decision Trees					0.9239	0.9215	0.8606	0.8601
Random Forest	0.989267	0.98767	0.82698	0.75305				

Extra Credit

We did the following extra credit tasks. Describe what you did as part of that task and present the corresponding results. Any source code you develop should be included in your Gradescope submission.

[QE.1] [Extra Credit Question #1] (10 Points)

You may earn up to 10 extra points if you analyze all four datasets using an additional algorithm. In other words, you would be evaluating more than two algorithms on each of the datasets discussed in Section 1.

[QE.2] [Extra Credit Question #2] (10 Points)

You may earn up to 10 extra points if you select a new challenging dataset and evaluate the performance of different algorithms on it. A challenging dataset should necessarily include both categorical and numerical attributes and have more than two classes; or it should be a dataset where you have to classify images.

[QE.3] [Extra Credit Question #3] (15 Points)

You may earn up to 15 extra points if you combine different algorithms and construct an ensemble. For instance, you could create an ensemble composed of three neural networks (each with a different architecture) and a random forest. The ensemble would be trained as described in class: each algorithm would be trained on its own bootstrap dataset, and so on. The final predictions would then be determined by majority voting. Evaluate your ensemble algorithm on all four datasets discussed in Section 1.

[QE.3] [Extra Credit Question #3] (15 Points)

You may earn up to 15 extra points if you implement a new type of algorithm—a non-trivial variant of one of the algorithms studied in class. If you choose to do so, you should evaluate its performance on all four datasets.